MambaScope: A Visual Tool for the Interactive Exploration of Mamba

Emma Bakker (12197971), Nina Delcaro (15828808), Owen de Jong (14311585)

***Abstract*—New deep learning architectures like Mamba, a selective state-space model (SSM), offer competitive performance to Transformers but lack inherent interpretability due to the absence of attention mechanisms. This opacity hinders understanding and comparison. We introduce MambaScope, a visual analytics system for the comparative interpretation of Mamba-based and Transformer-based vision models. MambaScope integrates coordinated views of layer-wise saliency maps, interactive UMAP projections of latent representations and CLIP-powered semantic search to facilitate exploration of model behavior. Through a comparative analysis of ViM-small and DeiT-small and a qualitative user study, we demonstrate that MambaScope effectively exposes ViM's internal focus patterns. Our evaluation shows the system enables users to uncover nuanced architectural strategies, such as ViM's more holistic processing versus DeiT's patch-focused approach, providing a needed framework for interpreting emerging deep learning models.**

I. Introduction

INTERPRETABILITY is essential for understanding, debugging and trusting deep learning models, especially as they are deployed in high-stakes and real-world settings [1], [2]. Interactive visualization has emerged as a powerful approach for exposing internal model behavior, revealing how inputs are processed and helping experts validate model decisions [3]–[5]. For Transformer architectures, attention maps offer a built-in interpretability signal and many visualization tools have been developed to help users inspect token-to-token relationships and attention flow [6]–[8].

As model design advances, however, new architectures are emerging that no longer expose such interpretable structures. A recent and promising example is *Mamba* [9], a selective state-space model (SSM) that replaces attention with input-conditioned recurrence. Mamba achieves strong performance on language, audio and vision tasks, including image classification with Vision Mamba (ViM) [10], while offering linear-time scaling and fast inference. Yet its internal representations are continuous and latent, lacking explicit token interactions or attention scores. This makes interpretation and comparison with Transformer-based models challenging, highlighting the need for tools that can support cross-architecture analysis.

In this paper, we introduce **MambaScope**, an interactive visualization system that enables comparison between Mamba-based and Transformer-based models on image classification tasks. MambaScope combines LRP-based saliency heatmaps, UMAP projections of internal representations and CLIP-based semantic retrieval in a coordinated, multi-view interface. It supports layer-wise exploration of how inputs are transformed and facilitates qualitative comparison of model focus and representation structure, without requiring attention mechanisms. Designed for ML researchers and expert users, MambaScope bridges the interpretability gap for emerging state-space models.

To summarize, this paper makes the following contributions:

- We introduce **MambaScope**, an interactive system for comparing how selective state-space models and Transformers process visual inputs at different layers.
- We integrate **layer-wise saliency maps, UMAP projections of internal representations and semantic retrieval** into a coordinated interface to highlight how each model encodes and propagates spatial information.
- We conduct a **comparative case study** using ViM-small and DeiT-small on a curated ImageNet subset, demonstrating how MambaScope reveals architecture-specific focus patterns and processing strategies.

II. Background

A. Mamba and Selective State-Space Models

State-space models (SSMs) are an alternative to attention-based models for handling sequences. Instead of comparing all tokens with each other like Transformers do, SSMs pass information step-by-step using a recurrent formula [11]. A typical update in an SSM looks like:

$$h_t = Ah_{t-1} + Bx_t, \tag{1}$$

where h_t is the hidden state at time t, x_t is the input token or image patch and A, B are learned matrices. These matrices stay the same across time steps, which makes the model efficient to run, but can limit how flexible it is.

Mamba [9] improves this setup by making the update depend on the input itself. In this *selective state-space model*, the matrices change with the input:

$$h_t = A(x_t)h_{t-1} + B(x_t)x_t. \tag{2}$$

This allows the model to control how much of the past to keep and how much of the current input to use at each step. It keeps the model fast, but makes it more expressive.

Mamba also adds a simple gating step to its output. This lets the model decide which parts of the SSM output to pass on:

$$y_t = \sigma(G(x_t)) \odot \mathrm{SSM}(x_t), \tag{3}$$

where $G(x_t)$ is a lightweight feedforward layer that outputs a gate vector, σ is a nonlinearity like sigmoid and $\odot$ denotes element-wise multiplication. Here, $\mathrm{SSM}(x_t)$ refers to the

hidden state update defined earlier and the gate helps filter out less relevant features while retaining those that are most informative.

In the vision domain, Vision Mamba (ViM) [10] applies this idea to image classification. It treats patches of the image as a sequence and uses SSMs instead of attention to process them across multiple layers.

Compared to Transformers, Mamba has a very different internal structure:

- Transformers use global attention maps that explicitly indicate how much each token attends to others. These attention weights are discrete and easy to visualize, which makes the model's internal focus interpretable.
- Mamba updates its hidden state through continuous transitions and input-conditioned gating. Since it does not produce token-to-token attention scores, its internal behavior is harder to visualize and interpret.

Because of this, it's harder to understand what Mamba is focusing on or how it reaches a decision. Unlike attention, which gives an interpretable signal, Mamba needs new tools to make its inner workings more transparent.

B. Models Compared

We compare two pretrained models of similar scale and accuracy to ensure a fair, interpretability-focused comparison: **ViM-small** [10], a 26M parameter SSM and **DeiT-small** [12], a 22M parameter Vision Transformer. Their closely matched performance on ImageNet (80.5% vs 79.9% top-1 acc.) makes their architectural differences the primary variable for our analysis.

III. Related Work

Understanding the internal behavior of deep models has become increasingly important as architectures grow in complexity and abstraction. Our work draws from and extends three main lines of prior research: methods for interpreting model internals, interactive tools for visual reasoning and techniques tailored to state-space models.

A. Visualizing Internal Representations

Numerous efforts have focused on making neural network internals accessible to human understanding. In the context of Transformers, visualizations often rely on attention maps to expose token-level relationships [8]. Tools like Attention-Viz [6] and InterpreT [7] extend this to show how attention patterns evolve across layers or heads. However, these techniques assume the presence of discrete, interpretable structures like attention matrices.

In response, other tools have visualized representations more globally. NeuroCartography [13] and Activation Atlas [14] use dimensionality reduction (e.g., UMAP, t-SNE) to map high-dimensional activations into spatialized 2D layouts. These views help uncover class clusters or representation shifts across layers. We adopt a similar embedding projection approach, extending it to support model-to-model comparison across layers and architectures.

B. Attribution and Saliency in Deep Models

Saliency methods aim to explain model predictions by identifying which input regions most influence the output. Prominent techniques include Layer-wise Relevance Propagation (LRP) [15] and Integrated Gradients [16], which produce heatmaps highlighting regions that contribute to a model's decision. These methods are widely used in vision tasks to provide localized, interpretable visualizations of model focus.

For Transformer-based vision models, Transformer-Attribution [17] adapts LRP to trace relevance through attention layers, enabling token-level saliency visualization. This has become a standard approach for interpreting attention-based architectures.

Recent work [18] extends interpretability to state-space models, showing that Mamba exhibits implicit attention patterns that can be captured using a modified LRP approach. Their *Mamba-Attribution* method adapts Transformer-Attribution to the recurrence-based computations of Mamba, enabling saliency extraction even in the absence of explicit attention weights.

We build directly on this technique by applying Mamba-Attribution to all intermediate layers of ViM. This allows us to generate layer-wise relevance maps that can be compared directly to those from DeiT, enabling side-by-side saliency analysis across architectures.

C. Interactive Visual Analytics for Model Understanding

Another line of research focuses on how user interaction can help make sense of complex models. Tools like ActiVis [3], CNNVis [5] and LSTMVis [4] let users explore model behavior through interactive features such as coordinated views, selection and linking across visual components. More recent systems [19], [20] provide multi-view interfaces for vision models that combine heatmaps, embedding plots and semantic labels. These tools help users understand where and how a model is focusing. MambaScope builds on this multi-view design approach but adapts it for state-space models, which lack explicit structures like attention maps or convolutional filters.

The design of MambaScope is also guided by high-level interaction frameworks in visual analytics. [21] distinguished between low-level interactions (e.g., selection) and high-level cognitive goals (e.g., explanation). [22] adapt this idea for foundation models, emphasizing how user action should support insight. We follow this perspective by structuring our interface to link user queries (via search, selection, layer switching) to interpretive tasks like comparison, focus validation and saliency reasoning.

D. Gaps in the Literature

Despite rich work in interpretability, existing systems overwhelmingly target attention- or convolution-based models. Little support exists for visualizing state-space models like Mamba, whose dynamics are continuous and latent. Moreover, while Mamba-Attribution enables saliency extraction, no tools allow layer-wise inspection or side-by-side comparison

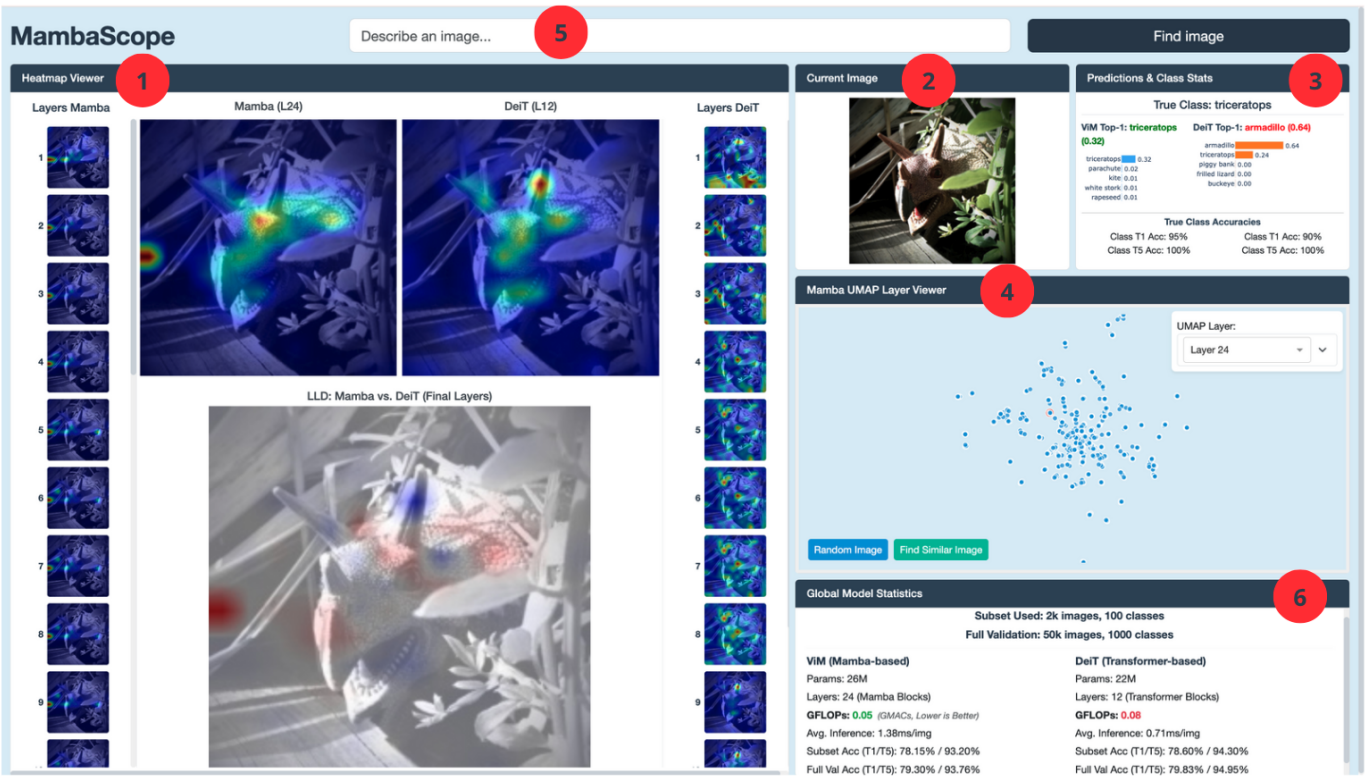

Fig. 1. The **MambaScope** user interface, comprising coordinated multi-view components: **(1) Heatmap Viewer** displays interactive layer-wise saliency maps from ViM and DeiT models, including a final-layer difference visualization; **(2) Current Image Panel** shows the full-resolution image selected by the user; **(3) Prediction & Class Statistics** provides detailed model predictions, confidence scores and class-level accuracies; **(4) UMAP Viewer** enables exploration and selection of ViM embeddings across different layers; **(5) Semantic Search Bar** facilitates concept-driven image retrieval using CLIP-based queries; **(6) Global Model Statistics** summarizes comparative metrics such as model size, GFLOPs, inference time and accuracy. These tightly integrated views collectively support detailed exploration, comparison and interpretation of Mamba and Transformer behavior across multiple layers.

with Transformer counterparts. Finally, global representation tools like UMAP projections have not been integrated into a coordinated visual interface that aligns architectures, saliency and semantic retrieval.

MambaScope addresses these gaps by providing an interactive system for layer-wise comparison between Mamba and Transformer models, integrating saliency maps, embedding projections and semantic search into a unified visual interface.

IV. Goals and Tasks

To ground the design of MambaScope, we followed a *design-study-lite* methodology [23] aimed at supporting the analytic sense-making processes of expert users like ML researchers and interpretability engineers [21]. Our first step was a focused literature review, synthesizing insights from both influential visualization tools for deep learning (e.g., ActiVis [3], NeuroCartography [13]) and cutting-edge explainability methods for the new class of state-space models [18]. This review enabled us to perform a task analysis, which we structured using a Why-What-How typology [24], to define the core goals and tasks that guide our system's design and subsequent evaluation.

A. Design goals

G1—Expose state-space internals. Allow analysts to *see* where Vision Mamba focuses and how its latent representations evolve, despite the absence of attention weights.

G2—Facilitate cross-architecture comparison. Align ViM and DeiT views so that differences in focus, abstraction and class separation can be inspected layer-by-layer.

G3—Support open-ended exploration. Provide tightly linked interactions that let hypotheses emerge, evolve and be verified in the fluid analytic loops characteristic of expert work.

B. Analytic tasks

The task descriptions below follow the Why–What–How lens: the *Why* states the analyst's motivation, the *What* defines data and operation and the *How* points to MambaScope's interface mechanism.

T1—Validate per-image focus across models and layers. *Why*: Establish whether a prediction relies on semantically plausible regions. *What*: Compare relevance heatmaps for DeiT and ViM produced by Transformer-Attribution and Mamba-Attribution, respectively, at arbitrary depths. *How*: A coordinated view presenting side-by-side saliency maps with interactive layer selection.

T2—Understand representation evolution. *Why*: Observe when class manifolds disentangle, merge or remain entangled. *What*: Inspect neighbourhood structure in 2D UMAP projections of per-layer embeddings. *How*: A layer-

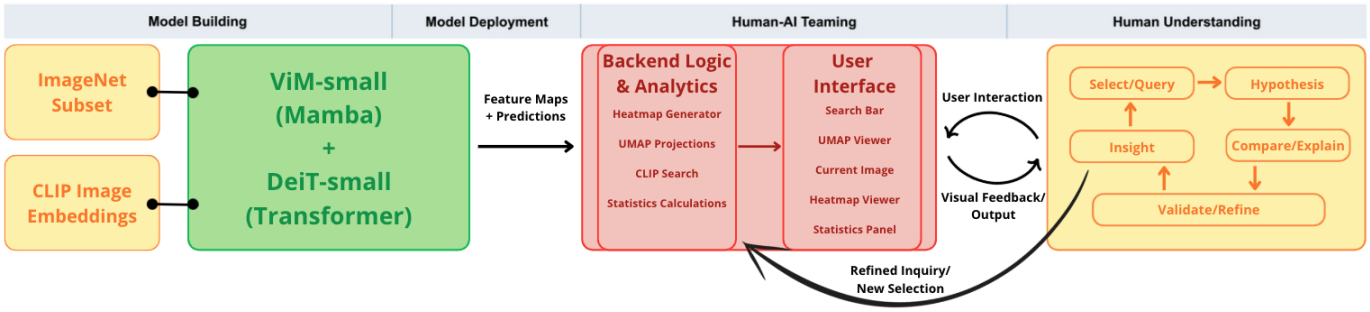

Fig. 2. System overview of MambaScope. The diagram follows the Multimedia Analytics for AI framework [22] and is adapted from their original Figure 1 to reflect our specific system components. It explicitly separates the low-level interface (User Interface) from the high-level reasoning loop (Human Understanding).

linked UMAP view whose points can be clicked to drive all other panels.

T3—Inspect model predictions for the selected image. *Why*: Compare what each architecture believes and how confident it is. *What*: Retrieve top-k labels and logits from ViM and DeiT for the currently selected image. *How*: A statistics panel that updates automatically when the user selects an image in UMAP or search results.

T4—Retrieve semantically relevant images. *Why*: Begin exploration with a concept (e.g. "striped cat") rather than a specific image. *What*: Encode a natural-language query with CLIP and rank the dataset by cosine similarity. *How*: A search bar whose thumbnail results can be clicked to populate the saliency and statistics views and highlighted in UMAP.

T5—Navigate depth to contextualise processing stages. *Why*: Determine which layer crystallises discriminative focus or representation structure. *What*: Traverse successive hidden-state snapshots and saliency maps. *How*: A thumbnail bar and depth slider that jointly update every coordinated view.

C. Coverage check

Together, T1–T5 span the analytic loop advocated in cognitive-interaction frameworks [21]. For instance, **T4** (semantic search) supports the initial explore phase, while **T1** (saliency validation) and **T3** (prediction inspection) facilitate the explain and validate stages. The ability to switch layers **(T5)** and navigate embeddings **(T2)** allows users to fluidly compare and refine hypotheses.

V. Sytem Design

MambaScope is an interactive visual analytics system designed to facilitate the exploration and comparative analysis of ViM and DeiT models in image classification tasks. The system's architecture, shown in Figure 2, is explicitly divided into backend analytics and a user-facing interface, following the Multimedia Analytics for AI framework [22]. This design supports an iterative human reasoning cycle by providing a responsive, interactive environment built upon precomputed analytical artifacts. This section elaborates on these components, aligning with the structure depicted in the diagram.

A. Backend Logic & Analytics

The backend is responsible for all heavy computation, generating the analytical artifacts that power the interactive frontend. By precomputing these artifacts, we ensure a fluid user experience. As shown in Figure 2, this layer processes model outputs to create the necessary data for comparison and exploration.

1) Heatmap Generator: To expose the internal focus of both models **(G1)**, this module generates per-layer saliency maps. For ViM, it uses *Mamba-Attribution* [18] a method adapted for state-space models. For DeiT, it uses the standard Transformer-Attribution [17]. These maps provide the raw data for visual focus comparison.

2) UMAP Projection module: To facilitate cross-architecture comparison of representation structure **G2**, this module computes 2D UMAP projections of ViM's patch embeddings for each layer. These projections spatialize the high-dimensional latent space, making it possible to visually inspect class separation and clustering.

3) CLIP-based Semantic Search: To support open-ended, concept-driven exploration **G3**, the backend indexes all dataset images using CLIP embeddings. A FAISS index is created to enable rapid, low-latency retrieval based on natural-language queries from the user.

4) Statistics Calculations: To provide quantitative context for every image, this module precomputes top-5 predictions, confidence scores and logits for both models. It also aggregates these to calculate class-level and global accuracy metrics, which are essential for grounding qualitative observations.

These backend modules continuously communicate with the UI, ensuring synchronized and responsive updates across all visual elements based on user actions.

B. User Interface

The MambaScope user interface (Figure 1) is the interactive layer where the user engages with the data generated by the backend. It integrates multiple coordinated views, each designed to realize the specific analytic tasks from Section IV. An interaction in one view, such as selecting an image, dynamically updates all other views.

1) Search Bar: To support **T4 (Retrieve semantically relevant images)**, this prominent input field allows users to initiate exploration from a high-level concept. It sends

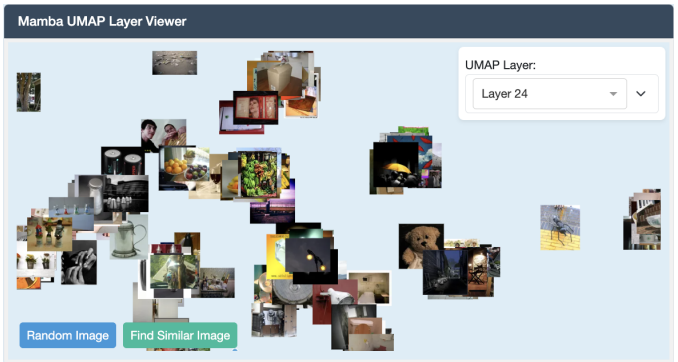

Fig. 3. Zoomed-in view of the UMAP Layer Viewer showing ViM patch embeddings. When zoomed, individual image thumbnails are displayed, facilitating visual inspection and comparison of localized clusters.

natural-language queries to the backend's CLIP-based Semantic Search module and populates the interface with the top result.

2) UMAP Viewer: This view operationalizes **T2 (Understand representation evolution)** and **T5 (Navigate depth)** by presenting an interactive visualization of ViM's per-layer embedding space. Users can pan and zoom to explore the global structure of the UMAP projections. Upon sufficient zoom, the data points resolve into image thumbnails, facilitating direct visual inspection of local clusters, as shown in Figure 3. Clicking any point or thumbnail selects it for a deep-dive analysis in the other coordinated views. A dropdown menu enables users to switch between layers, revealing how ViM's representational geometry evolves with model depth.

Two additional interactive functionalities enhance exploration capabilities:

- **Random Image**: Retrieves and highlights a random image from the current UMAP view, facilitating unbiased exploration and serendipitous discovery.
- **Find Similar Image**: Identifies and highlights an image most semantically similar to the currently selected image, enabling targeted exploration based on visual and semantic similarity.

3) Current Image View.: Serving as a constant visual anchor, this panel displays the full-resolution image currently under investigation. It provides essential context for interpreting the saliency maps **(T1)** and understanding the model predictions **(T3)**.

4) Heatmap Viewer: As the primary tool for **T1 (Validate per-image focus)**, this viewer presents the saliency maps from the backend's Heatmap Generator. It features a side-by-side comparison for any layer, a final-layer difference map and interactive thumbnail galleries for quick navigation through model depth **T5**.

5) Statistics Panel: To fulfill **T3 (Inspect model predictions)**, this panel displays detailed quantitative outputs from the Statistics Calculations module for the selected image. It includes top-5 predicted classes, confidence scores and bar charts of the logits from both models, contextualized by class-level accuracy metrics.

6) Global Model Statistics: Supporting the high-level goal of cross-architecture comparison **(G2)**, this panel provides a concise summary of key model metrics like parameter count, GFLOPs and inference speed. It allows analysts to immediately frame their qualitative findings within the context of model complexity and efficiency. To further illustrate these statistics and facilitate clearer interpretation, we summarize the results in Table I.

Metric	ViM-small	DeiT-small
Parameters (M)	26	22
GFLOPs (GMACs)	0.05	0.08
Avg. Inference (ms/img)	1.38	0.71
Subset Top-1 Accuracy (%)	78.15	78.60
Subset Top-5 Accuracy (%)	93.20	94.30
Full Validation Top-1 Acc. (%)	79.30	79.83
Full Validation Top-5 Acc. (%)	93.76	94.95

TABLE I
GLOBAL MODEL STATISTICS COMPARING VIM-SMALL AND DEIT-SMALL ON THE CURATED IMAGENET SUBSET (2K IMAGES, 100 CLASSES) AND THE FULL VALIDATION SET (50K IMAGES, 1000 CLASSES).

C. Interaction Workflow

The backend and UI work together to support the iterative analytic workflow conceptualized in the "Human Understanding" loop of Figure 2. This process, inspired by cognitive-interaction frameworks [21], [22], enables fluid, open-ended inquiry **(G3)** through the following stages:

1) Select/Query: A user initiates exploration by selecting an image from the UMAP plot **(T2)** or submitting a semantic query like "striped cat" ((**T4**).

2) Hypothesize: Initial observations from the saliency maps or embedding clusters lead to hypotheses. For example, the user might notice that ViM's saliency is more diffuse while DeiT's is sharply focused.

3) Compare/Explain: The user uses the side-by-side views to compare the saliency patterns **(T1)**, prediction confidences **(T3)** and embedding structures **(T2)** across models and layers.

4) Validate/Refine: Hypotheses are tested interactively. The used might switch layers **(T5)** to see where the focus diverges or select images from the UMAP view to check if the pattern holds.

5) Insight: Through this loop, the analyst builds a validated understand of architectual differences.

For example, a user might *select* an image of a 'tench' (a type of fish), *hypothesize* that ViM focuses more on water texture while DeiT hones in on the fish's outline, *compare* their saliency maps **(T1)** and then *validate* this by searching for 'fish in murky water' **(T4)** to see if the pattern holds across similar images. This cyclic workflow fosters continuous exploration and sensemaking, operationalizing the core goals of making latent space-state internals interpretable (**G1**), facilitating direct cross-architecture comparisons (**G2**) and enabling fluid, open-ended inquiry (**G3**).

VI. IMPLEMENTATION DETAILS

This section details the technical realization of MambaScope, complementing the conceptual system architecture previously discussed (Section V) with specific implementation choices and parameters. Our open-source code is available at: https://github.com/Kingkumar1999/MambaScope.

A. Dataset and Models

1) Dataset: We use a curated dataset of the ImageNet-1k validation set [25], created by randomly sampling 100 classes and selecting 20 images per class, for a total of 2,000 images. All images are preprocessed by resizing 224×224 pixels and normalizing according to standard ImageNet statistics.

2) Models: We compare two pretrained models chosen for their architectural contrast yet comparable performance: Vision Mamba (**ViM-small**) [9], a state space model with 26M parameters and a Data-Efficient Image Transformer (**DeiT-small**) [12], a Vision Transformer with 22M parameters. Their similar complexity and accuracy make them an ideal pair for a fair, interpretability-focused comparison.

B. Backend Implementation

To ensure a responsive user experience, all computationally intensive analytical artifacts are precomputed and stored. The backend was implemented in Python using PyTorch and computations were performed on a cluster node with an NVIDIA A100 GPU.

1) Saliency Maps: We generate layer-wise relevance maps for both models using LRP-based methods, which attribute a model's prediction back to its input features.

For **DeiT-small**, we apply *Transformer-Attribution* [17]. This method computes a relevance matrix $\bar{\mathbf{A}}^{(b)}$ for each Transformer block b by combining attention weights with the gradient signal:

$$\bar{\mathbf{A}}^{(b)} = \mathbf{I} + \mathbb{E}_h \left[\left(\nabla_{\mathbf{A}^{(b)}} \odot \mathbf{R}^{(n_b)} \right)^+ \right], \tag{4}$$

Here, $\mathbf{R}^{(n_b)}$ is the relevance propagated back from the subsequent layer, essentially weighting the attention gradients by what was seen as important downstream. The identity matrix $\mathbf{I}$ crucially accounts for skip-connections, preserving relevance flow across layers. The final relevance map for the model is a composition of the block-wise matrices.

For **ViM-small**, which has no explicit attention maps, we use *Mamba-Attribution*, a recently proposed variant tailored to state-space architectures [18]. This method infers token interactions from the model's internal dynamics. The layer-wise relevance $\hat{\boldsymbol{\beta}}^{(\lambda)}$ is calculated as:

$$\hat{\boldsymbol{\beta}}^{(\lambda)} = \mathbf{I} + \left(\nabla_{\hat{y}^{(\lambda)}} \odot \hat{\boldsymbol{\alpha}}^{(\lambda)} \right)^+, \tag{5}$$

Here, $\hat{\boldsymbol{\alpha}}^{(\lambda)}$ is the *hidden attention map*, an inferred matrix that captures token-level influence based on Mamba's input-dependent recurrence. This allows us to extract an interpretable relevance signal comparable to that of a Transformer.

To generate a saliency map for a specific layer l, the relevance matrices of preceding layers are composed via matrix multiplication. For DeiT, we aggregate all layers from the start: $C_t^{(l)} = \bar{\mathbf{A}}^{(1)} \cdot \bar{\mathbf{A}}^{(2)} \cdots \bar{\mathbf{A}}^{(l)}$. For ViM, following the methodology of Mamba-Attribution which emphasizes the importance of later layers, we adapt their approach for intermediate analysis. The relevance for any layer l is computed by composing the matrices of the preceding nine blocks (or fewer if $l < 9$): $C_m^{(l)} = \bar{\mathbf{A}}^{(\max(1,l-8))} \cdots \bar{\mathbf{A}}^{(l)}$. This sliding-window approach enables layer-wise inspection while respecting the model's architectural properties.

To directly visualize differences between the models, we compute a final-layer **Latent Layer Difference** map:

$$\text{LLD} = \frac{(\text{MambaRelevance} - \text{DeiTRelevance}) + 1}{2} \tag{6}$$

This normalizes the difference to the $[0, 1]$ range. In our visualization, this allows us to use a diverging colormap (e.g., blue-white-red), where blue indicated stronger attribution by DeiT, red by ViM and white indicates no differences.

2) UMAP Embeddings: For each of ViM's 24 layers, we extract path embeddings for all 2,000 images. We then apply UMAP with $n_neighbors = 15$, $min_dist = 0.15$ and a cosine metric to generate 2D projections. The resulting coordinates are stored in a Parquet file alongside image metadata.

3) Model Predictions: We run both models on all images to precompute top-1 and top-5 predictions, softmax probabilities and correctness flags. These are aggregated to derive class-level and overall model accuracies.

4) Semantic Indexing: To power the search functionality, we encode all 2,000 images using the public CLIP VIT-B/32 model. The resulting 512-dimensional embeddings are indexed using FAISS with a flat index and cosine similarity, enabling low-latency retrieval at runtime.

5) Interative UMAP Exploration Buttons: To facilitatie intuitive exploration, two interactive operations were integrated into the backed logic.

- *Random Image:* Selects and return a random image from the available dataset, implemented through lightweight sampling from the cached dataset metadata, allowing users to discover insights without explicit search queries.
- *Find Similar Image:* Retrieves an image embedding most similar to the currently selected image by querying the precomputed FAISS index. This operation directly leverages the semantic indexing backend, enabling targeted exploration based on semantic similarity and visual proximity in the embedding space.

C. Frontend Implementation

The user interface is a web application built entirely in Python using **Dash** and **Dash Bootstrap Components**.

1) Visualization: All interactive plots, including the UMAP scatter plot, are rendered using **Plotly's scattergl mode**, which leverages WebGL for hardware accelaration to ensure smooth navigation even with thousands of data points. Saliency maps are overlaid on images using Matplotlib and served as base64-encoded strings.

2) Interactivity: To assess MambaScope's effectiveness in supporting its design goals (G1-G3) and analytical tasks (T1-T5) and to evaluate its overall usability, we conducted a qualitative evaluation with target users.

VII. Evaluation

To assess whether MambaScope effectively supports its design goals and to gather usability feedback, we conducted

a qualitative evaluation. We recruited six participants representative of our target audience: one domain expert (an MIT assistant professor specializing in Explainable AI) and five graduate students in Artificial Intelligence. This sampling strategy is consistent with prior studies that use small, targeted cohorts to evaluate visual analytics systems intended for expert use [26], [27].

Participants were provided with access to the MambaScope application and a structured questionnaire. Following best practices in visualization evaluation [28], [29], we used a mixed-methods design: 5-point Likert-scale questions [30] captured quantitative ratings, while open-ended questions elicited rich, qualitative insights. The questionnaire first guided participants through the tool's features before presenting evaluation tasks.

A. Findings on Design Goals

G1: Exposing State-Space Internals. Participants strongly agreed that MambaScope made ViM's internal workings more interpretable, giving this a high average rating of 4.5/5 ($\sigma = 0.55$). The UMAP viewer was key, with users noting they could see "class-level clusters begin to emerge" in later layers. The professor described it as a "compelling visual proxy for the representational hierarchy learned by ViM." The layer-wise heatmaps were also highlighted as crucial for providing a "basic understanding" of ViM's focus. This feedback confirms the system's success in demystifying the novel architecture.

G2: Facilitating Cross-Architecture Comparison. The system proved effective for direct comparison. The ease of using the heatmaps (including the LLD map) to discern focus differences was rated 4.3/5 ($\sigma = 1.21$). Users' confidence in their ability to describe the models' different visual strategies after using the tool was 3.5/5 ($\sigma = 1.38$). Multiple users independently formed the hypothesis that ViM employs a "smoother" or "more holistic" focus, while DeiT is "more patch-focused" on "sharp details," demonstrating that the tool enables the discovery of nuanced architectural behaviors.

G3: Supporting Open-Ended Exploration. The system's ability to support a hypothesis-driven workflow received mixed but positive feedback. The combination of search/selection and the statistics panel was rated as highly useful for targeted comparisons (4.7/5, $\sigma = 0.52$). However, the overall support for forming and testing hypotheses was rated 3.5/5 ($\sigma = 1.38$), indicating that while the components are effective, their integration into a fluid analytical loop could be improved. The CLIP search relevance was seen as adequate for initiating exploration (3.3/5, $\sigma = 0.52$).

B. Usability and Feedback

Overall usability was rated favorably at 4.3/5 ($\sigma = 0.82$). The Search Bar (4.8/5) and Heatmap Viewer (4.5/5) were noted as particularly user-friendly. Participants provided actionable feedback, with a recurring suggestion to add UMAP projections for DeiT to allow for direct comparison of representational geometries. One user suggested making the LLD map more interactive, such as "highlight[ing] that same part on the other two heatmaps" when hovering. This feedback validates MambaScope's core utility while providing a clear direction for future refinement.

VIII. Discussion

The emergence of high-performing yet opaque architectures like Mamba necessitates new interpretability tools that can look beyond established mechanisms like attention. MambaScope was designed to bridge this gap, guided by the principle that effective cross-architecture comparison requires linking local explanations (saliency) with global context (embedding structure and quantitative metrics).

Our evaluation confirms the value of this integrated design. While a single saliency map may be ambiguous, MambaScope allows users to contextualize it by exploring the representational space via UMAP or retrieving semantically similar instances via CLIP. This ability to fluidly move between different levels of abstraction proved crucial for users in forming and validating hypotheses, such as the key finding that ViM develops a more *holistic* focus compared to DeiT's *patch-focused* strategy. This demonstrates that a well-coordinated system can empower users to uncover nuanced architectural behaviors that are not apparent from accuracy metrics alone.

However, our system has important limitations. Its reliance on precomputation, while ensuring interactivity, limits scalability to the thousands of images used in our subset. Analyzing larger datasets would require a more complex on-demand computation architecture. Furthermore, the quality of our visual explanations is fundamentally dependent on the underlying attribution methods; as more faithful techniques for state-space models are developed, they could be integrated to improve MambaScope's insights. Finally, UMAP can distort global distances and should be treated as a powerful heuristic for hypothesis generation rather than a ground-truth map of the latent space.

IX. Conclusion and Future Work

In this paper, we introduced MambaScope, a new interactive visual analytics system for the comparative interpretability of Vision Mamba and Vision Transformer models. By tightly integrating layer-wise saliency maps, UMAP projections of latent space and semantic search into a coordinated multi-view interface, MambaScope provides a tool for understanding emerging state-space architectures. Our qualitative evaluation confirmed that the system enables experts to effectively explore ViM's internal mechanisms, compare its processing strategies directly against DeiT and generate new, verifiable hypotheses about their architectural differences.

Building on our findings and user feedback, we identify several key directions for future work. First, we plan to enhance direct comparison by incorporating UMAP projections for DeiT, allowing for a side-by-side analysis of representational geometry. Second, we will explore more advanced interactions, such as linked highlighting between the LLD map and its source heatmaps, to further lower the cognitive load of comparison. Finally, we aim to address the current scalability limitations by investigating a computation model that combines precomputation with on-demand analytics, paving the way for the analysis of larger-scale models and datasets.

References

[1] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv: Machine Learning*, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:11319376

[2] Z. C. Lipton, "The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery." *Queue*, vol. 16, no. 3, p. 31–57, Jun. 2018. [Online]. Available: https://doi.org/10.1145/3236386.3241340

[3] M. Kahng, P. Y. Andrews, A. Kalro, and D. H. P. Chau, " ActiVis: Visual Exploration of Industry-Scale Deep Neural Network Models ," *IEEE Transactions on Visualization & Computer Graphics*, vol. 24, no. 01, pp. 88–97, Jan. 2018. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TVCG.2017.2744718

[4] H. Strobelt, S. Gehrmann, H. Pfister, and A. M. Rush, " LSTMVis: A Tool for Visual Analysis of Hidden State Dynamics in Recurrent Neural Networks ," *IEEE Transactions on Visualization & Computer Graphics*, vol. 24, no. 01, pp. 667–676, Jan. 2018. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TVCG.2017.2744158

[5] M. Liu, J. Shi, Z. Li, C. Li, J. Zhu, and S. Liu, " Towards Better Analysis of Deep Convolutional Neural Networks ," *IEEE Transactions on Visualization & Computer Graphics*, vol. 23, no. 01, pp. 91–100, Jan. 2017. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TVCG.2016.2598831

[6] C. Yeh, Y. Chen, A. Wu, C. Chen, F. Viégas, and M. Wattenberg, "Attentionviz: A global view of transformer attention," 2023. [Online]. Available: https://arxiv.org/abs/2305.03210

[7] E. Aflalo, M. Du, S.-Y. Tseng, Y. Liu, C. Wu, N. Duan, and V. Lal, "Vl-interpret: An interactive visualization tool for interpreting vision-language transformers," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 21 374–21 383.

[8] J. Vig, "A multiscale visualization of attention in the transformer model," *ArXiv*, vol. abs/1906.05714, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:189762556

[9] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2024. [Online]. Available: https://arxiv.org/abs/2312.00752

[10] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision mamba: Efficient visual representation learning with bidirectional state space model," 2024. [Online]. Available: https://arxiv.org/abs/2401.09417

[11] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," Oct. 2021. [Online]. Available: http://arxiv.org/abs/2111.00396v3

[12] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers amp; distillation through attention," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 10 347–10 357. [Online]. Available: https://proceedings.mlr.press/v139/touvron21a.html

[13] H. Park, N. Das, R. Duggal, A. P. Wright, O. Shaikh, F. Hohman, and D. H. Chau, "Neurocartography: Scalable automatic visual summarization of concepts in deep neural networks," 2021. [Online]. Available: https://arxiv.org/abs/2108.12931

[14] S. Carter, Z. Armstrong, L. Schubert, I. Johnson, and C. Olah, "Activation atlas," *Distill*, vol. 4, no. 3, p. e15, 2019.

[15] S. Bach, A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, and W. Samek, "On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation," *PLoS ONE*, vol. 10, no. 7, p. e0130140, 7 2015. [Online]. Available: https://doi.org/10.1371/journal.pone.0130140

[16] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML'17. JMLR.org, 2017, p. 3319–3328.

[17] H. Chefer, S. Gur, and L. Wolf, "Transformer interpretability beyond attention visualization," 2021. [Online]. Available: https://arxiv.org/abs/2012.09838

[18] A. Ali, I. Zimerman, and L. Wolf, "The hidden attention of mamba models," 2024. [Online]. Available: https://arxiv.org/abs/2403.01590

[19] Y. Li, J. Wang, X. Dai, L. Wang, C.-C. M. Yeh, Y. Zheng, W. Zhang, and K.-L. Ma, "How does attention work in vision transformers? a visual analytics attempt," 2023. [Online]. Available: https://arxiv.org/abs/2303.13731

[20] J. Wang, C.-C. M. Yeh, U. S. Saini, and M. Das, "Visual attention exploration in vision-based mamba models," 2025. [Online]. Available: https://arxiv.org/abs/2502.20764

[21] W. A. Pike, J. Stasko, R. Chang, and T. A. O'Connell, "The Science of Interaction," *Information Visualization*, vol. 8, no. 4, pp. 263–274, 1 2009. [Online]. Available: https://doi.org/10.1057/ivs.2009.22

[22] M. Worring, J. Zahálka, S. van den Elzen, M. T. Fischer, and D. A. Keim, "A multimedia analytics model for the foundation model era," 2025. [Online]. Available: https://arxiv.org/abs/2504.06138

[23] M. Sedlmair, M. Meyer, and T. Munzner, "Design study methodology: Reflections from the trenches and the stacks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 18, no. 12, pp. 2431–2440, 2012.

[24] M. Brehmer and T. Munzner, "A multi-level typology of abstract visualization tasks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 19, no. 12, pp. 2376–2385, 2013.

[25] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

[26] D. Gotz, J. Zhang, W. Wang, J. Shrestha, and D. Borland, "Visual analysis of high-dimensional event sequence data via dynamic hierarchical aggregation," *IEEE transactions on visualization and computer graphics*, vol. 26, no. 1, pp. 440–450, 2019.

[27] P. Klacansky, H. Miao, A. Gyulassy, A. Townsend, K. Champley, J. Tringe, V. Pascucci, and P.-T. Bremer, "Virtual inspection of additively manufactured parts," in *2022 IEEE 15th Pacific Visualization Symposium (PacificVis)*. IEEE, 2022, pp. 81–90.

[28] P. Saraiya, C. North, and K. Duca, "An insight-based methodology for evaluating bioinformatics visualizations," *IEEE transactions on visualization and computer graphics*, vol. 11, no. 4, pp. 443–456, 2005.

[29] S. Wang, H. Yan, K. E. Isaacs, and Y. Sun, "Visual exploratory analysis for designing large-scale network-on-chip architectures: A domain expert-led design study," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 4, pp. 1970–1983, 2023.

[30] A. Joshi, S. Kale, S. Chandel, and D. K. Pal, "Likert scale: Explored and explained," *British journal of applied science & technology*, vol. 7, no. 4, p. 396, 2015.